

# Advanced Database Concepts

## Entity-Relationship (ER) Modeling

### Definition:

ER Modeling is a technique to **visually represent the data structure of a system**, showing entities (things), their attributes, and the relationships between them.

Think of it as the **blueprint of your database**.

### 1 Core Components

| Component           | Description                                    | Example                      |
|---------------------|------------------------------------------------|------------------------------|
| <b>Entity</b>       | A real-world object or concept with data       | User, Order, Product         |
| <b>Attribute</b>    | A property of an entity                        | name, email, created_at      |
| <b>Primary Key</b>  | Unique identifier for an entity                | user_id                      |
| <b>Relationship</b> | How entities are connected                     | User places Order            |
| <b>Foreign Key</b>  | Attribute linking related entities             | order.user_id → user.user_id |
| <b>Cardinality</b>  | Number of entity occurrences in a relationship | 1:1, 1:N, N:M                |

## What is Normalization?

**Normalization = Organizing database tables to reduce redundancy and avoid data anomalies.**

### Goal:

- **No duplicate data**
- **Easy updates**
- **Better data consistency**

**Think of it like:**

**Instead of storing the same information many times, store it once and reference it.**

## **Why Normalization Matters**

**Without normalization you get problems:**

| <b>Problem</b> | <b>Example</b> |
|----------------|----------------|
|----------------|----------------|

|                       |                                                          |
|-----------------------|----------------------------------------------------------|
| <b>Update anomaly</b> | <b>Changing a user's email requires updating 10 rows</b> |
|-----------------------|----------------------------------------------------------|

|                       |                                                           |
|-----------------------|-----------------------------------------------------------|
| <b>Insert anomaly</b> | <b>Can't insert a department until an employee exists</b> |
|-----------------------|-----------------------------------------------------------|

|                       |                                                             |
|-----------------------|-------------------------------------------------------------|
| <b>Delete anomaly</b> | <b>Deleting a user accidentally deletes department info</b> |
|-----------------------|-------------------------------------------------------------|

**Normalization fixes this.**

### **1. Update Anomaly**

**This happens when the same information is stored in many places.**

**Example:**

**Imagine a database where every time a user makes an order, the system stores:**

- order info**
- product**
- user email**

**So if a user has made 10 orders, their email is stored 10 times.**

**Now the user changes their email.**

**The system must update 10 different records.**

**If one record is missed, the database now has two different emails for the same user.**

**So the data becomes inconsistent.**

**Normalization fixes this by storing the email only once (in the user record) and referencing it elsewhere.**

## **2. Insert Anomaly**

**This happens when the database doesn't allow you to add certain information unless other unrelated information exists.**

**Example:**

**Suppose the database stores employees and departments in the same structure.**

**Now the company creates a new department (HR), but no employees are hired yet.**

**Because the system expects employee information, you cannot store the department yet.**

**So you are forced to either:**

- **leave the department unrecorded**
- **or insert fake employee data**

**Both are bad.**

**Normalization separates departments and employees, so a department can exist even if no employees are assigned yet.**

## **3. Delete Anomaly**

**This happens when removing one piece of data accidentally deletes other important information.**

**Example:**

**Suppose a database stores employee details and department information together.**

**If a department currently has only one employee and that employee leaves the company, deleting that employee record will also remove the only record containing the department information.**

Now the database has lost the department entirely, even though the department still exists.

Normalization prevents this by storing department information separately from employee records.

## Data Redundancy

Data Redundancy means the same data is stored multiple times in a database.

### Example

If a user's email is stored in every order record, the email is repeated many times.

### Why it's bad

- Wastes storage
- Causes inconsistent data (if one copy is updated but others aren't)
- Leads to update, insert, and delete anomalies

### One-line interview answer

Data redundancy is the unnecessary duplication of the same data in multiple places in a database.

## The Important Normal Forms (80% for interviews)

You mostly need 1NF → 3NF.

---

## 1NF (First Normal Form)

### Rule

- No repeating groups
- Each column contains atomic values

✗ Bad

| user_id | phones  |
|---------|---------|
| 1       | 123,456 |

✓ Good

| user_id | phone |
|---------|-------|
| 1       | 123   |
| 1       | 456   |

Mental model

One cell = One value

## 2NF (Second Normal Form)

Rule

- Must be in 1NF
- No partial dependency on composite primary key

Example:

Primary key = (student\_id, course\_id)

✗ Bad

| student_id | course_id | student_name |
|------------|-----------|--------------|
| 1          | C1        | Rahul        |

student\_name depends only on student\_id.

✓ Fix

Student Table

| student_id | name  |
|------------|-------|
| 1          | Rahul |

**Enrollment Table**

| student_id | course_id |
|------------|-----------|
| 1          | C1        |

**Mental model**

Column must depend on entire primary key, not just part.

### 3NF (Third Normal Form)

**Rule**

- Must be in 2NF
- No transitive dependency

**✗ Bad**

| user_id | city   | zipcode |
|---------|--------|---------|
| 1       | Mumbai | 400001  |

**But:**

zipcode → city

City depends on zipcode, not user.

**✓ Fix**

**Users**

| user\_id | zipcode |

**Zipcodes**

| zipcode | city |

**Mental model**

Columns should depend only on the primary key.

## **Real World Note (Important)**

**In real systems we sometimes break normalization for performance.**

**That is called:**

**Denormalization**

**Example:**

- **storing user\_name in orders table to avoid JOIN.**

## **1. Normalization vs Denormalization**

### **Normalization**

- **Splitting data into multiple related tables**
- **Reduces data redundancy**
- **Improves data consistency**
- **Uses JOINS to fetch data**

**Example:**

**Users table + Orders table**

### **Denormalization**

- **Intentionally combining data into fewer tables**
- **Reduces JOINS**
- **Improves read performance**
- **May introduce data redundancy**

**Example:**

**Store user\_name directly inside orders.**

### **One-line interview answer**

**Normalization reduces redundancy and improves data integrity, while denormalization intentionally duplicates data to improve read performance.**

## **2. BCNF (Boyce-Codd Normal Form)**

**BCNF is a stronger version of 3NF.**

**Rule:**

**For every dependency  $A \rightarrow B$ , A must be a candidate key.**

**Meaning:**

**Only keys should determine other attributes.**

**Why it exists:**

**Sometimes a table satisfies 3NF but still has redundancy, so BCNF fixes those edge cases.**

**Short interview answer**

**BCNF is an advanced normal form where every determinant must be a candidate key.**

## **3. Does Normalization Improve Performance or Consistency?**

**Normalization mainly improves data consistency, not performance.**

**Because:**

- **Data stored once**
- **Fewer update errors**
- **Better data integrity**

**But it can sometimes reduce performance because queries require JOINS.**

**Interview answer**

**Normalization improves data consistency and integrity, but may reduce read performance due to additional JOIN operations.**

## **4. Why Companies Sometimes Denormalize**

**Companies denormalize mainly for performance and scalability.**

**Reasons:**

- 1. Reduce expensive JOINS**

2. Improve read speed
3. Optimize high-traffic queries
4. Better for analytics / reporting

**Example:**

**In an e-commerce system:**

- Instead of joining users every time,
- store user\_name inside orders.

**Interview answer**

**Companies denormalize to reduce JOINS and improve read performance in high-traffic systems.**

**What is the Cost of Joins?**

**When a query uses JOIN, the database must:**

1. Read multiple tables
2. Match rows using keys
3. Combine results

**This requires extra CPU, memory, and time.**

**So joins are computationally expensive, especially with large datasets.**

## Isolation

Isolation is a property of database transactions that ensures multiple transactions running at the same time do not interfere with each other.

It makes each transaction behave as if it is running alone in the system, even though many transactions are happening concurrently.

---

### Simple idea

If two users update or read the same data at the same time, isolation controls what each transaction can see so the data stays correct.

### Why Isolation is Needed

Isolation is needed because many users access and modify the database at the same time.

Without isolation, transactions could interfere with each other, leading to incorrect or inconsistent data.

### The 4 Isolation Levels

Ordered from least strict → most strict.

READ UNCOMMITTED

READ COMMITTED

REPEATABLE READ

SERIALIZABLE

Higher level = more safety but slower performance.

#### 1. READ UNCOMMITTED

Lowest isolation.

A transaction can see uncommitted changes from another transaction.

This causes Dirty Reads.

Example:

- Transaction A updates a balance to 500
- Transaction B reads 500
- Transaction A rolls back

Now the real value was never 500, but B already read it.

**Problem**

**Dirty reads.**

**Reality**

**Most databases don't even allow this.**

## **2. READ COMMITTED**

**A transaction can only read committed data.**

**So dirty reads cannot happen.**

**But another problem can happen:**

**Non-repeatable read**

**Example:**

**Transaction A reads a price = 100**

**Another transaction updates price → 150**

**Transaction A reads again → 150**

**Same query → different result.**

**Used by**

**Many systems including default in PostgreSQL.**

## **3. REPEATABLE READ**

**Once a transaction reads a row, it will always see the same value during that transaction.**

**Even if another transaction updates it.**

**Example:**

**Transaction A reads price = 100**

**Transaction B updates → 150**

**Transaction A reads again → still 100**

**Database uses snapshots (MVCC).**

**But another issue can happen:**

**Phantom reads**

**Example:**

**Transaction A reads all orders > 100**

**Later another transaction inserts a new order = 200**

**Now result set changes.**

#### **4. SERIALIZABLE**

**Highest isolation.**

**Transactions behave as if they run one by one.**

**No anomalies allowed.**

**So:**

- **No dirty reads**
- **No non-repeatable reads**
- **No phantom reads**

**But downside:**

- **Slowest**
- **More locks / conflicts**

**Used in critical financial systems.**

## 1. Row-Level Locking

Row-level locking means the database locks only the specific row being modified.

Example idea:

User A updates order #101.

Only that row is locked.

Other users can still:

- read or update order #102
- update order #103

So multiple operations can happen at the same time.

Why it's good

- High concurrency
- Better performance in busy systems

One-line interview answer

Row-level locking locks only the specific row being modified, allowing other rows in the table to be accessed concurrently.

## 2. Table-Level Locking

Table-level locking means the database locks the entire table when one operation runs.

Example:

User A updates one order.

But now the entire orders table is locked.

Other users must wait before reading or updating anything in that table.

Problems

- Low concurrency
- Slower in high traffic systems

**Why some DBs use it**

- **Simpler implementation**
- **Sometimes faster for very small tables**

**One-line interview answer**

**Table-level locking locks the entire table during an operation, preventing other transactions from accessing it.**

### **3. Deadlocks**

**A deadlock happens when two transactions wait for each other forever.**

**Example flow:**

**Transaction A**

- **locks row 1**
- **wants row 2**

**Transaction B**

- **locks row 2**
- **wants row 1**

**Now:**

**A waits for B**

**B waits for A**

**Both are stuck.**

**This is a deadlock.**

**What databases do**

**Most databases detect deadlocks and automatically kill one transaction.**

**One-line interview answer**

**A deadlock occurs when two or more transactions hold locks and wait for each other indefinitely.**

## 1. Pessimistic Concurrency/locking

**Assumption:**

**Conflicts are likely, so prevent them early.**

**The system locks the data immediately when someone starts modifying it.**

**Example flow:**

1. Transaction A reads a row
2. It locks the row
3. Transaction B tries to update it
4. B must wait until A finishes

**So conflicts are prevented using locks.**

**Used in**

- Banking systems
- Financial transactions
- Inventory systems

**One-line interview answer**

**Pessimistic concurrency assumes conflicts are likely and uses locks to prevent other transactions from modifying the same data.**

## 2. Optimistic Concurrency

**Assumption:**

**Conflicts are rare, so don't lock immediately.**

**Instead:**

1. Transactions read data freely
2. When updating, the system checks if data changed meanwhile

**Usually done with a version number or timestamp.**

**Example:**

**Row version = 5**

**Transaction A reads version 5**

**Transaction B updates → version becomes 6**

**Now A tries to update.**

**System checks:**

**expected version = 5**

**actual version = 6**

**Conflict detected → update fails.**

**What happens next in real systems?**

**Usually the application:**

- 1. Detects the failure**
- 2. Reads the latest row again**
- 3. Retries the update**

**This is why optimistic concurrency works well when conflicts are rare.**

**A must retry.**

**Used in**

- Web apps**
- High scalability systems**
- APIs**

**One-line interview answer**

**Optimistic concurrency assumes conflicts are rare and checks for conflicts during updates instead of locking data.**

**What is MVCC?**

**MVCC = a database technique that allows multiple versions of a row to exist so reads and writes don't block each other.**

**Instead of locking rows heavily, the database creates new versions of data when updates happen.**

**So:**

**Old version → kept**

**New version → created**

**Different transactions can read different versions safely.**

---

## **Why MVCC Exists**

**Without MVCC:**

- **Reads might wait for writes**
- **Writes might wait for reads**

**That slows everything down.**

**MVCC allows:**

- **Readers to keep reading old data**
- **Writers to update data simultaneously**

**This gives high concurrency.**

---

## **Simple Example**

**Suppose a row looks like this.**

**balance = 100**

**version = 1**

**Step 1 — Transaction A reads**

**Transaction A reads:**

**balance = 100**

---

## **Step 2 — Transaction B updates**

**Transaction B updates the balance to:**

**balance = 150**

**version = 2**

**The database does not overwrite immediately.**

**Instead:**

**Version 1 → balance 100**

**Version 2 → balance 150**

---

## **Step 3 — Transaction A continues**

**Transaction A still sees:**

**balance = 100**

**Because its snapshot started earlier.**

**New transactions see:**

**balance = 150**

---

## **Key Idea (Mental Model)**

**Think of MVCC like time travel snapshots.**

**Each transaction sees the database at the moment it started.**

**Transaction A → sees old snapshot**

**Transaction B → sees new snapshot**

**So they don't block each other.**

---

## **Where MVCC is Used**

**Many modern databases use it:**

- **PostgreSQL**
- **MySQL (InnoDB engine)**

- Oracle Database
  - SQLite
- 

## Why MVCC is Powerful

### Benefits:

- Readers don't block writers
  - Writers don't block readers
  - High database concurrency
  - Fewer locks
- 

## One-Line Interview Answer

MVCC allows multiple versions of a row so transactions can read consistent snapshots without blocking writes.

## What are Batch Inserts / Bulk Operations?

Batch inserts mean inserting many rows in a single database operation instead of one-by-one.

This reduces the number of database calls.

---

## Without Batch Insert (Slow)

Application sends one query per row.

Example idea:

Insert user1

Insert user2

Insert user3

Insert user4

If you insert 1000 rows, that means 1000 database queries.

Problems:

- High network overhead
  - More transaction cost
  - Slower performance
- 

### **With Batch Insert (Fast)**

All rows are inserted in one query.

Example idea:

Insert user1, user2, user3, user4

Now 1000 rows → 1 query.

This is much faster.

---

### **Why Batch Inserts Are Faster**

1. Fewer network round trips
2. Single transaction
3. Less query parsing
4. Better disk write efficiency

Databases handle large writes more efficiently in chunks.

### **Common Use Cases**

Batch operations are used in:

- Data migration
- CSV imports
- Analytics pipelines
- Logging systems
- Background jobs

## 1. Stored Procedures

A Stored Procedure is a pre-written SQL program stored inside the database that can be executed when needed.

Instead of sending many SQL queries from the application, you call one procedure.

Example idea

Instead of doing this from the app:

```
INSERT user  
UPDATE account  
INSERT log
```

You create a stored procedure:

```
create_user()
```

Then just call:

```
CALL create_user()
```

The database runs all the queries internally.

---

### Why Stored Procedures are Used

1. Better performance (precompiled SQL)
  2. Less network communication
  3. Reusable logic
  4. Better security (users call procedure instead of direct table access)
- 

### One-line Interview Answer

A stored procedure is a precompiled set of SQL statements stored in the database that can be executed as a single call.

## 2. Triggers

A Trigger is a piece of code that automatically runs when a specific database event happens.

Events can be:

- INSERT
- UPDATE
- DELETE

The database fires the trigger automatically.

---

### Example

Suppose you want to log every user deletion.

When someone deletes a user:

**DELETE FROM users**

A trigger automatically runs and:

**INSERT INTO audit\_log**

So the log happens without the application doing anything.

---

### Common Uses

Triggers are used for:

- Audit logs
  - Automatic timestamps
  - Data validation
  - Maintaining derived data
- 

### One-line Interview Answer

A trigger is a database function that automatically executes when a specific event like INSERT, UPDATE, or DELETE occurs.

## Deadlock

A deadlock happens when two or more transactions wait for each other's locks, and none can proceed.

### How Databases Detect Deadlocks

Most databases build a Wait-for Graph

Example:

T1 → waiting for T2

T2 → waiting for T1

If a cycle appears, the DB detects a deadlock.

Databases like PostgreSQL and MySQL automatically detect this.

### How Databases Resolve Deadlocks

When detected:

The database kills one transaction.

Example:

ERROR: deadlock detected

transaction rolled back

The killed transaction is called the victim transaction.

Then the application should retry the transaction.

### How to Prevent Deadlocks (Real World)

**1** Lock resources in the same order

Bad:

T1: users → orders

T2: orders → users

Good:

T1: users → orders

T2: users → orders

---

**2 Keep transactions short**

Long transactions = more chance of lock conflicts.

---

**3 Use proper indexes**

Without indexes, DB may lock more rows than needed.

---

**4 Avoid unnecessary locks**

Only lock what you need.